Project 2: Neighborhood-based / Model-based Recommendation

추천시스템 · 2025-27567 배문성 · 2026년 10월

1. 데이터와 평가 방식

`ratings_train.csv`는 80,036건, 사용자 547명 × 영화 7,357편으로 밀도가 1.99%이고 평균 평점은 3.5483이다. 영화당 평점 수의 중앙값은 3개이며, 평점이 1개뿐인 영화가 2,267편(31%)이다. user_id와 movie_id는 연속하지 않으므로 정렬된 고유 id를 0부터 시작하는 행/열 인덱스로 매핑해 547×7357 행렬을 만들었다.

`ratings_val.csv` 10,071건 가운데 사용자와 영화가 모두 train에 있는 행(warm)은 **1,405건, 사용자 20명**뿐이다(84명 중 64명이 train에 없음). 명세에 따라 cold-start 행은 RMSE에서 제외했고, 이하 "val RMSE"는 이 1,405건 기준이다. 다만 `main.py`는 어떤 입력에도 값을 내야 하므로 train에 없는 사용자나 영화는 영화 평균 → 사용자 평균 → 전역 평균 순으로 대체한다.

Task 3의 하이퍼파라미터는 **train을 무작위로 90/10 분할(seed 0)한 내부 홀드아웃**(8,111건 중 warm 7,851건)에서만 골랐다. val은 선택에 쓰지 않고 결과 확인에만 사용했다(명세 3-b). 보고서의 모든 수치는 제출한 `main.py`의 함수를 그대로 호출해 측정했다.

2. Task 1: User-based CF

평점 행렬 R(미평가 0), 평가 여부 마스크 C, 평균 보정 행렬 $D = (R - \bar{r}_u) \circ C$를 만든다. $\bar{r}_u$는 사용자 u가 평가한 모든 영화의 평균이고, 미평가 칸은 D에서 0이다. 덕분에 공통 평가 항목 I_{uv}에 대한 합이 행렬곱으로 바로 계산된다.

$$\mathrm{Num} = DD^\top, \quad A = (D \circ D)C^\top, \quad s(u,v) = \frac{\mathrm{Num}_{uv}}{\sqrt{A_{uv}A_{vu}}}$$

$A_{uv} = \sum_{i \in I_{uv}} (r_{ui} - \bar{r}_u)^2$이므로 분모의 두 합도 명세대로 I_{uv}로 제한된다. 공통 항목이 없거나 분산이 0이면 $s = 0$이고, 대각은 0으로 두어 $v \neq u$를 보장했다. 예측은

$$\hat{r}_{ui} = \bar{r}_u + \frac{\sum_v s(u,v) D_{vi}}{\sum_v |s(u,v)|\, C_{vi}}$$

이며, 영화 i를 평가하지 않은 v는 $D_{vi} = C_{vi} = 0$이라 자동으로 빠진다. 분모가 0이면(모든 유사도 0) 영화 평균을 쓴다. 명세의 계산을 재현하는 것이 목적이므로 clip이나 top-k 이웃 제한은 넣지 않았다.

$\bar{r}_u$를 I_{uv} 위의 평균으로 바꾼 변형도 시험했다. val RMSE가 0.9429 → 0.9495로 나빠졌고 기준 사용자 세 명도 모두 나빠져(0.6304 / 0.6901 / 0.7169), 사용자 전체 평균을 택했다.

3. Task 2: Matrix Factorization (SVD)

빈칸을 각 영화의 평균 평점으로 채운 행렬 F를 `np.linalg.svd(F, full_matrices=False)`로 분해한다(U: 547×547, 특이값 547개, $V^\top$: 547×7,357). 큰 특이값 $K = 400$개만 남겨 user factor $U_K\Sigma_K$(547×400)와 item factor $V_K^\top$(400×7,357)를 matmul 해 예측 행렬을 복원한다. 분해는 한 번만 하고, 입력 쌍마다 복원 행렬에서 값을 읽는다. Task 1과 같은 이유로 clip은 하지 않았다. 두 Task 모두 명세의 사용자별 기준을 만족한다(그림 1).

4. Task 3: Optimization

4.1 최적화 대상: Task 2

Task 2는 사용자별 기준은 통과하지만 전체 val RMSE가 1.0836으로, **모든 쌍에 전역 평균을 내는 것(1.0727)보다도 나쁘다.** 원인은 결측 대치다. F의 98%가 영화 평균으로 채운 값이라 첫 번째 특이값 하나가 에너지의 99.87%를 차지하고, F는 사실상 "영화 평균 행렬"이 된다. 그림 2처럼 K를 키우면 train 관측 평점은 거의 외우지만(K=400에서 RMSE 0.0783), val RMSE는 K와 거의 무관하게 1.063–1.084에 머물고 **영화 평균만으로 예측한 값(1.0840)과 사실상 같다**(K=400 예측과 영화 평균의 차이는 평균 0.0076). 그림 3에서 오차가 가장 큰 구간은 train 평점이 1개인 영화(1.2722)다. 이런 영화의 "평균"은 평점 한 개 그 자체여서 아무 보정 없이 예측으로 쓰인다.

즉 Task 2의 문제는 K 같은 파라미터가 아니라 **대치값을 학습하는 구조**에 있다. 그래서 (1) 관측된 평점에만 손실을 걸고, (2) 평점이 적은 영화·사용자의 효과를 정규화로 줄이는 방향으로 Task 2를 고쳤다.

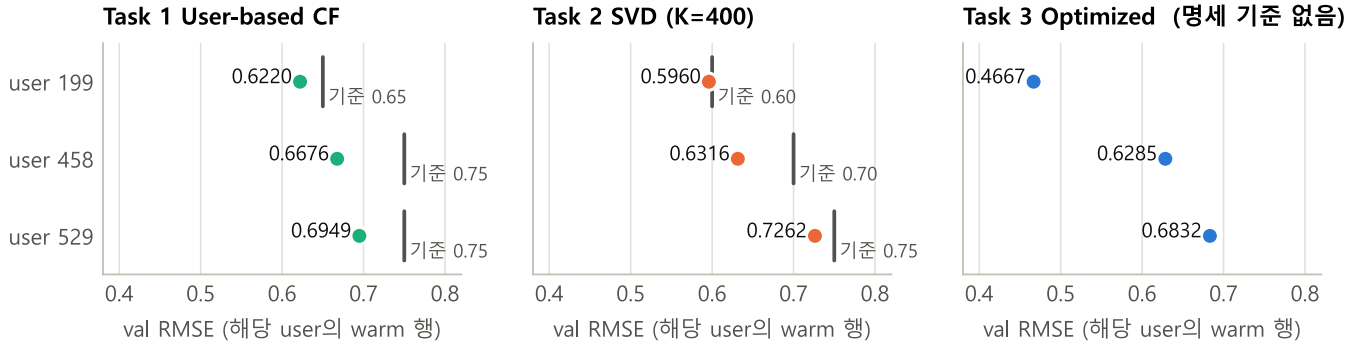


그림 1: 명세의 사용자별 검증 기준. 검은 세로선이 기준값이다. Task 1·2 모두 기준을 만족하며, Task 2의 user 199(0.5960)가 기준 0.60에 가장 가깝다.

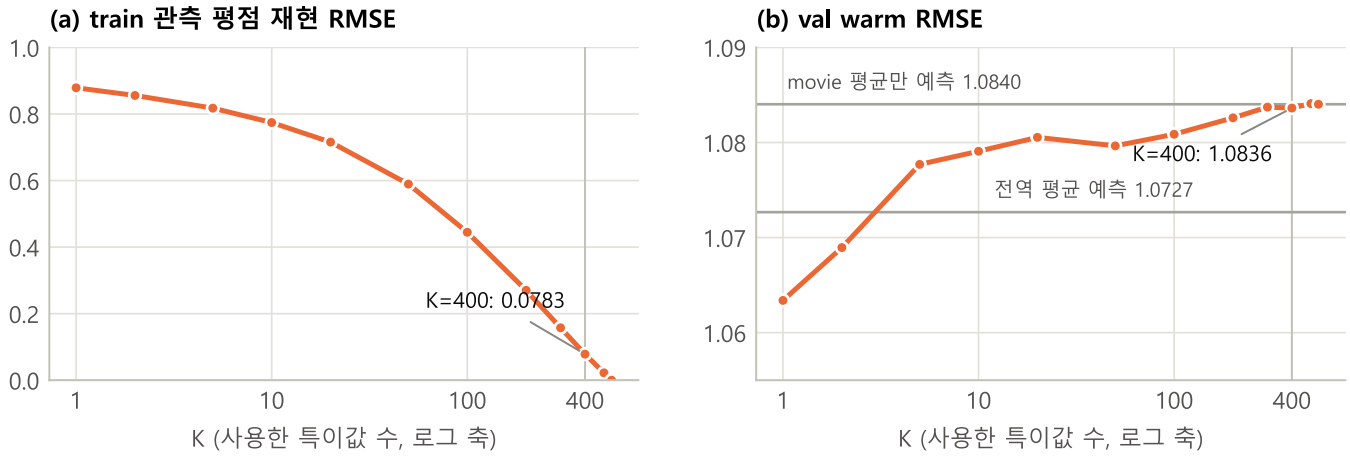


그림 2: Task 2에서 K에 따른 변화. (a) K가 커질수록 train 관측 평점을 외우지만, (b) val RMSE는 거의 변하지 않고 영화 평균만 쓴 예측(위쪽 회색 선)에 붙는다.

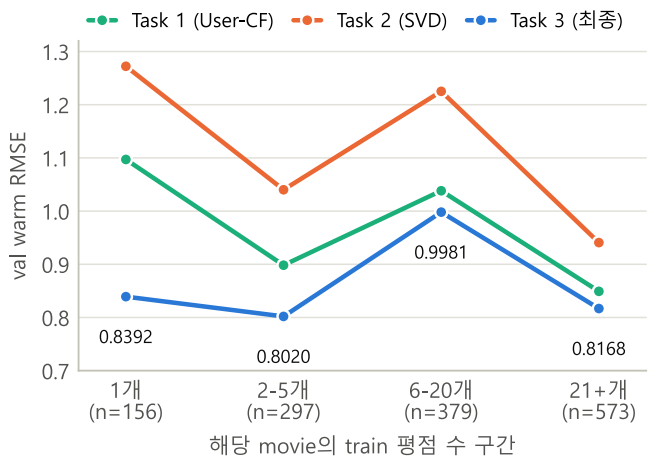


그림 3: 영화의 train 평점 수 구간별 val RMSE. Task 2는 평점이 적은 영화에서 가장 크게 틀리고, Task 3은 그 구간을 가장 크게 개선한다. 숫자는 Task 3의 값이다.

4.2 주요 로직

최종 예측식은 다음과 같다($w = 0.2$).

$$\hat{r}_{ui} = \mathrm{clip}_{[0.5,5]}\big((1-w)(\mu + b_u + b_i + p_u^\top q_i) + w\,\hat{r}_{ui}^{\mathrm{CF}}\big)$$

1. **정규화된 baseline.** 관측된 평점 집합 $\mathcal{K}$에 대해 다음을 최소화한다.

$$\sum_{(u,i)\in\mathcal{K}} (r_{ui} - \mu - b_u - b_i)^2 + \lambda_b \left(\sum_u b_u^2 + \sum_i b_i^2 \right)$$

   닫힌 해 $b_i = \sum_{u\in U_i}(r_{ui} - \mu - b_u)/(|U_i| + \lambda_b)$와 같은 꼴의 b_u 식을 15회 번갈아 적용했다. $\lambda_b = 3$이면 평점이 1개인 영화의 편차는 1/4로 줄어든다. Task 2의 가장 큰 오차 원인을 직접 겨냥한 단계다.
2. **잠재 요인.** baseline의 잔차에 $p_u, q_i \in \mathbb{R}^{100}$을 학습한다. 손실은 관측된 80,036건에만 걸고 L2 정규화($\lambda = 0.1$)를 둔다. 미니배치 SGD(배치 4,096, 학습률 0.01, 60 epoch, 초기값 $\mathcal{N}(0, 0.1^2)$, seed 42)로 학습하며, numpy만으로 벡터화했다.
3. **User-based CF 블렌딩.** 전역 저차원 구조를 쓰는 MF와 국소 이웃을 쓰는 Task 1은 틀리는 방식이 달라, 소량을 섞으면 오차가 일부 상쇄된다.
4. **clip.** 평점 범위 $[0.5, 5]$ 밖의 예측을 자른다.

train에 없는 사용자·영화는 Task 1과 같은 평균값으로 대체한다. 100쌍 입력 기준 `main.py` 전체 실행 시간은 약 16초다.

4.3 하이퍼파라미터 선택

후보마다 seed 0–4로 다섯 번 학습해 **평균 홀드아웃 RMSE**로 비교했다(그림 4). 처음에는 seed 42 한 번의 결과로 $K = 10$을 골랐다. 그런데 seed만 바꿔도 홀드아웃 RMSE가 0.8859(seed 42)에서 평균 0.8905 ± 0.0025로 나빠져, 그 선택이 seed 운에 좌우됐음을 확인하고 기준을 바꿨다.

- **K와 λ.** $\lambda = 0.1$에서 K가 커질수록 좋아지다가 $K = 100$에서 가장 낮고(0.8844 ± 0.0010), $K = 200$(0.8848)과는 차이가 없다. $\lambda = 0.05$는 모든 K에서 더 나쁘고 seed 편차도 크다(최대 ± 0.0040). $\lambda = 0.2$는 K에 따른 변화가 거의 없고 seed 편차가 0.0003 이하여서, 잠재 요인이 정규화에 눌려 거의 학습되지 않는 것으로 보인다.
- **나머지 네 값.** $K = 100$에 두고 하나씩 바꿨다. $\lambda_b = 0$이 +0.0156으로 가장 민감하고(정규화 없는 baseline은 Task 2의 영화 평균 문제를 다시 만든다), 3–5에서 평탄하게 가장 낮다. 학습률 0.02(−0.0010), 150 epoch(−0.0013), $w = 0.1$(−0.0001)은 홀드아웃에서 조금 낮았지만 모두 seed 표준편차(약 0.001) 수준이라 기존 값을 유지했다(150 epoch는 학습 시간도 2.5배). Task 1만 쓰는 $w = 1$은 0.9262로, 블렌딩은 소량일 때만 이득이다.

그림 4의 회색 val 곡선도 홀드아웃과 같은 모양이어서, 홀드아웃으로 고른 값이 val에서도 유지된다.

4.4 결과 분석

그림 5은 제출 설정(seed 42)에서 구성 요소를 하나씩 더한 결과다. 최종 모델은 Task 2 대비 홀드아웃 0.9924 → **0.8848**, val 1.0836 → **0.8689**로 낮아졌고 Task 1(0.9279 / 0.9429)보다도 낮다.

- **개선의 대부분은 ① 정규화된 baseline에서 나온다**(Task 2 대비 홀드아웃 −0.094, val −0.206). 4.1의 진단대로 대치값 대신 관측값만 쓰고 영화 효과를 줄이는 것이 핵심이었다.
- ② 잠재 요인은 홀드아웃 −0.0130, val −0.0069를 더한다. ③ clip은 범위를 벗어나는 예측이 드물어 효과가 거의 없고(홀드아웃 −0.0001, val −0.0002), ④ 블렌딩은 홀드아웃 −0.0010, val −0.0012를 더한다. 두 평가 집합에서 단계별 개선의 방향이 같다.
- 그림 3에서 Task 2 대비 개선은 **평점이 1개인 영화에서 가장 크다**(1.2722 → 0.8392). 이 구간에서 최종 모델은 baseline(0.8397)과 거의 같은데, 정보가 없는 영화에서는 정규화된 b_i가 사실상 전부이기 때문이다. 반대로 평점이 21개 이상인 영화에서는 잠재 요인이 baseline 대비 0.8312 → 0.8168로 추가 개선을 낸다. 잠재 요인은 데이터가 있는 곳에서만 기여하고, 데이터가 없는 곳은 정규화가 맡는 의도대로 동작한다.
- 기준 사용자 세 명(그림 1)에서도 0.4667 / 0.6285 / 0.6832로 Task 1·2보다 모두 낮다. train 평점이 18개뿐인 user 458은

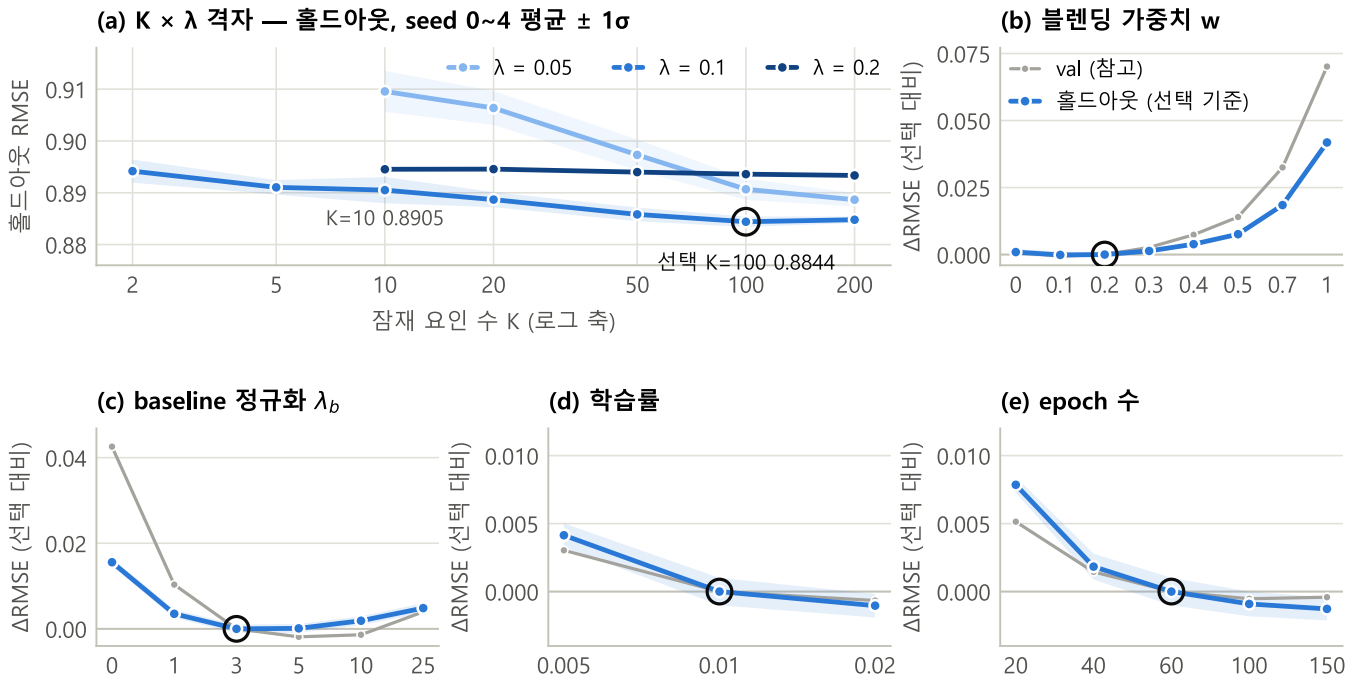


그림 4: Task 3 하이퍼파라미터. 모든 점은 seed 0–4 평균이고, 띠는 ± 1σ, 검은 원은 선택한 값이다. (b)–(e)는 선택 설정 대비 RMSE 차이로, 홀드아웃(파랑)과 val(회색)을 같은 기준선 0에 맞췄다.

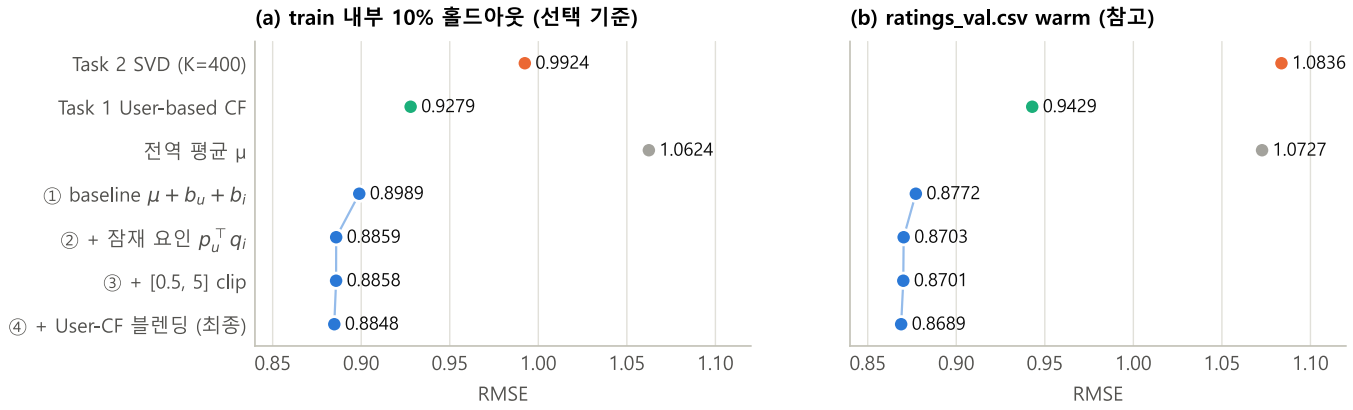


그림 5: Task 3 구성 요소를 하나씩 더한 결과(제출 설정, seed 42). 홀드아웃과 val에서 순서와 방향이 같고, 개선의 대부분은 ① 정규화된 baseline에서 나온다.

baseline만으로 0.6394이고, 잠재 요인과 블렌딩으로 0.6285까지 내려간다.

- **seed의 영향.** 제출 seed 42의 val 0.8689는 seed 평균 0.8664 ± 0.0016보다 조금 높다. 반대로 이전 설정($K = 10$)은 seed 42에서 val 0.8661로 더 낮게 나오지만 seed 평균은 0.8709다. 채점용 test에 대한 기대 성능은 단일 seed보다 평균이 더 잘 나타내므로, 평균 기준으로 고른 $K = 100$이 더 나은 선택이다.

한계. val warm 사용자가 20명뿐이라 사용자 단위 해석은 조심스럽고, train에 없는 사용자·영화는 단순 평균으로만 대체한다. 장르 같은 부가 정보가 없으므로 cold-start는 이 데이터로 더 개선하기 어렵다.

5. 피드백

- **평가 데이터.** val의 84명 중 64명이 train에 없어, 실제로 쓸 수 있는 val은 1,405건(20명)뿐이었다. 이 규모로는 하이퍼파라미터 차이를 판단하기 어려워 train 내부 홀드아웃을 따로 만들었다. 사용자가 train과 겹치도록 val을 나누면 학생이 성능을 가늠하기 쉬울 것 같다.
- **Task 1 수식의 $\bar{r}_u$.** 사용자 전체 평균인지 공통 평가 항목 위의 평균인지 명시해 주면 좋겠다. 두 해석의 결과가 다르고(val 0.9429 vs 0.9495), 채점이 "허용 범위 내 오차"로 이루어지기 때문이다.
- **Task 2의 $K = 400$.** 547 중 400을 쓰면 train을 외워 전역 평균보다 나쁜 결과가 나온다. 처음에는 구현 오류를 의심했지만, 그 원인을 분석한 과정이 Task 3의 방향을 잡는 데 가장 도움이 됐다.
- **기준 사용자 RMSE.** 세 사용자의 RMSE 기준은 구현이 명세와 맞는지 바로 확인할 수 있어 디버깅에 유용했다.